END-TERM PROJECT-01 REPORT

ON

CAR DODGING GAME

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412J2

Course Title & Code: (CSE)

Submitted By: Gatikrushna Bhuyan

Registration No: 2341002188

Guided By:Saurav Sir

Academic Year: 2023-2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the design and development of a **2D Car Dodging Game** implemented using C++ and the SFML (Simple and Fast Multimedia Library). The main objective of this project is to create an interactive game where the player controls a car and avoids collisions with incoming enemy vehicles.

The game simulates a vertical road environment where enemy cars continuously appear from the top and move downward. The player must navigate left and right to avoid these obstacles. The score increases based on survival time, and the game difficulty gradually increases by increasing speed and traffic density.

This project demonstrates the application of object-oriented programming concepts, real-time event handling, collision detection, and graphical rendering. It also enhances logical thinking, problem-solving skills, and understanding of game development fundamentals.

_____ _____

Submitted By Guided By

TABLE OF CONTENTS

LIST OF FIGURES

01. Introduction

Game development is one of the most engaging applications of programming, combining logic, mathematics, graphics, and real-time user interaction. With the advancement of computer graphics and multimedia libraries, it has become easier to design interactive applications and games using high-level programming languages such as C++.

This project focuses on the design and development of a **2D Car Dodging Game** using C++ and the SFML (Simple and Fast Multimedia Library). The game simulates a real-time driving environment where the player controls a car moving along a vertically scrolling road. The primary objective of the game is to avoid collisions with incoming enemy vehicles while surviving for as long as possible.

The player can control the car using keyboard inputs to move left and right across different lanes. Enemy cars are generated dynamically at random positions and move downward toward the player. As the game progresses, the difficulty increases by speeding up the enemy vehicles and increasing their frequency. This creates a challenging and engaging gameplay experience.

02. Problem Statement

The problem addressed in this project is to design and develop a **2D Car Dodging Game** that simulates a real-time driving environment where the player must avoid collisions with continuously incoming vehicles. The system should provide smooth gameplay, accurate collision detection, and responsive controls to ensure a good user experience.

The key challenges involved in this problem include:

- Handling real-time user input for controlling the player's car
- Generating enemy cars dynamically at random positions
- Ensuring smooth movement of objects across the screen
- Detecting collisions accurately between player and enemy cars
- Maintaining and displaying score based on survival time
- Increasing game difficulty progressively to keep the player engaged
- Managing memory efficiently by removing off-screen objects
- Providing a visually clear and interactive interface

The goal is to create a smooth, responsive, and engaging gameplay experience.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the **Car Dodging Game using C++ and SFML**. The objectives of this practical project are:

• To develop a working 2D game using the C++ programming language and SFML library.
• To understand and apply Object-Oriented Programming concepts such as classes and inheritance.

• To implement real-time user input handling using keyboard controls.
• To design and implement collision detection between game objects.
• To develop a scoring system based on player survival time.
• To implement dynamic difficulty by increasing speed and obstacle frequency.
• To enhance programming, debugging, and problem-solving skills.
• To follow a structured approach for writing efficient and maintainable code.
• To gain practical exposure to real-time graphical application development.

04. Tools and Technologies Used

The following tools and technologies were used in this project:

Programming Language: C

Compiler: GCC / Turbo C / Dev-C++

Development Environment: CodeBlocks / VS Code

Operating System: Windows / Linux

These tools provide a suitable environment for writing, compiling, and executing C programs.

05. Methodology

• Understanding the Problem: Identifying the game requirements such as player control, enemy generation, collision detection, scoring system, and increasing difficulty.

• System Design: Designing the structure of the game using Object-Oriented Programming concepts with classes like Car, PlayerCar, EnemyCar, and Game.

• Algorithm Design: Creating a step-by-step logical solution for game flow, including movement, collision detection, and scoring.

• User Input Handling: Implementing keyboard controls (Left/Right or A/D keys) to allow smooth player movement within the road boundaries.

• Enemy Car Generation: Generating enemy cars at random positions and intervals to create dynamic and unpredictable gameplay.

• Movement Logic: Updating positions of player and enemy cars based on speed and time to ensure smooth motion.

• Collision Detection: Checking for intersection between player and enemy cars using bounding box technique and ending the game on collision.

• Score System: Increasing the score based on survival time or number of dodged cars and displaying it on the screen.

• Difficulty Adjustment: Gradually increasing game difficulty by increasing speed and reducing time between enemy car spawns.

• Memory Management: Removing enemy cars that move off-screen to optimize performance and memory usage.

• Program Development**:** Writing the code in C++ using the SFML library to implement player control, enemy movement, collision detection, and scoring system.

• Compilation and Execution**:** Compiling the program using a suitable compiler (MinGW/GCC) and executing the game to verify functionality and performance.

• Testing and Debugging: Identifying and fixing logical, runtime, and graphical errors to ensure smooth gameplay.

• Performance Optimization: Improving efficiency by reducing unnecessary computations and ensuring stable frame rates.

• Final Output: Ensuring the game runs correctly with proper controls, visuals, scoring system, and increasing difficulty.

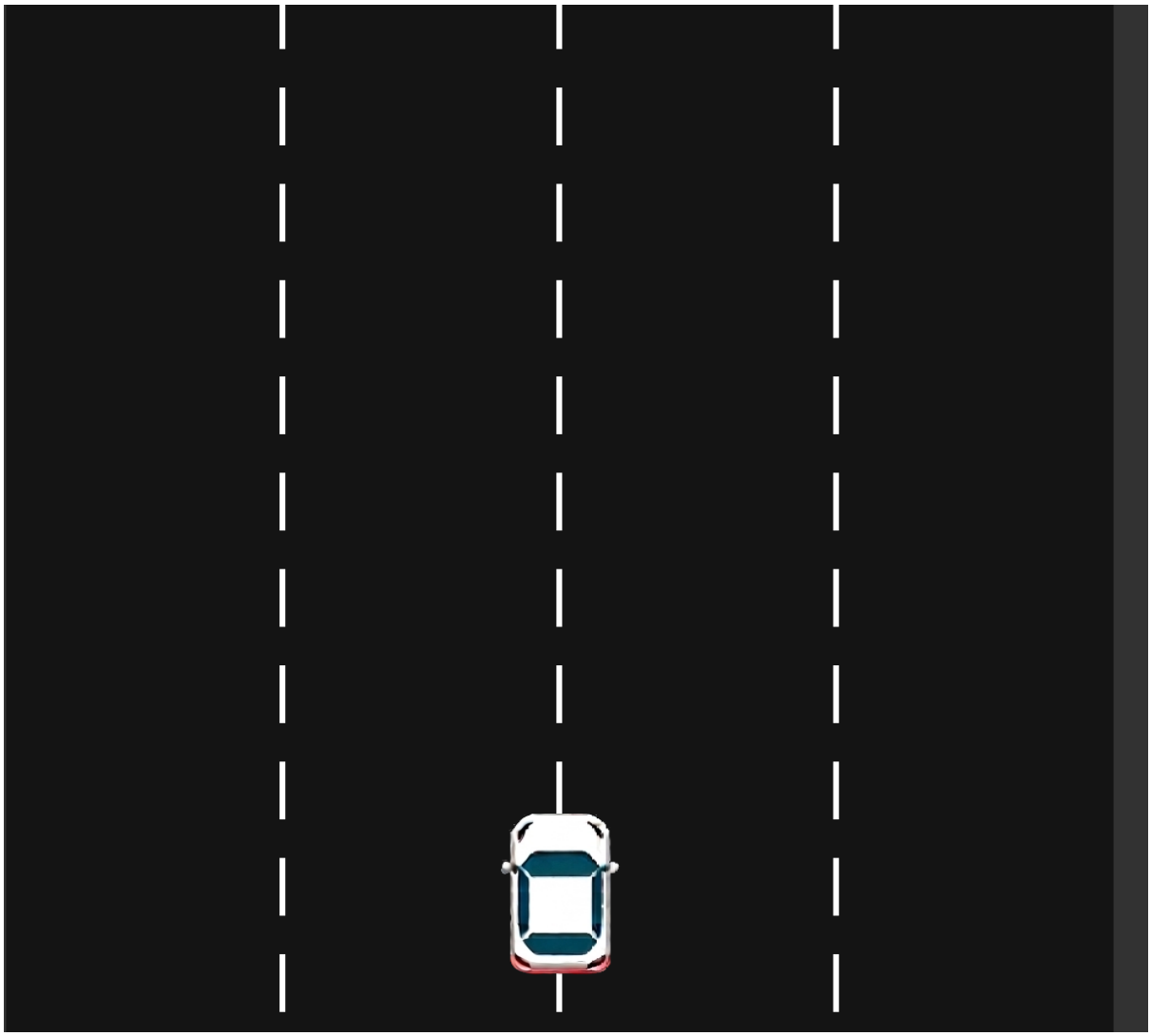
06. Results Analysis and Screenshots

The game was successfully implemented and tested. The player can control the car using keyboard inputs, and enemy cars spawn dynamically.

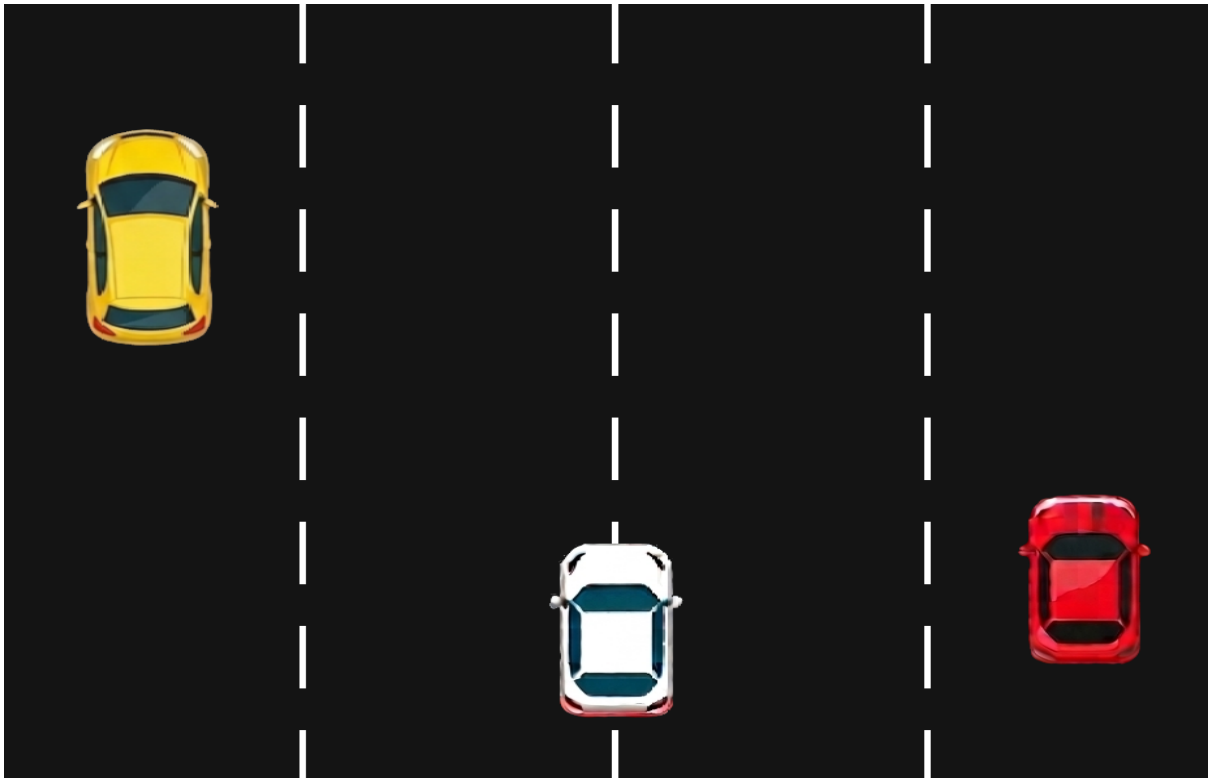
Key observations:

- Smooth movement and rendering
- Accurate collision detection
- Increasing difficulty over time
- Score updates correctly

GAME STARTS



CAR MOVING



GAME ENDS

07. Logical Understanding

The logic of the program is based on:

• Taking input from the user through keyboard controls to move the player car left and right

• Continuously running a game loop to update the game state in real-time

• Generating enemy cars at random positions using conditions and timers

• Updating positions of player and enemy cars using movement logic and speed variables

• Detecting collisions between player car and enemy cars using bounding box technique

• Processing game conditions such as score increase and game over using conditional statements

• Applying functions and classes for modular and structured coding

• Rendering game objects like road, cars, and score on the screen using graphics library

• Displaying the final output including score and game status clearly to the user

This project helps in understanding real-time processing, control flow, and logical execution in game development using C++.

08. Conclusion

The project was successfully completed using the **C++ programming language and SFML library**. It strengthened the understanding of important concepts such as object-oriented programming, real-time input handling, collision detection, and graphical rendering.

The project provided practical experience in developing an interactive game application and improved problem-solving, debugging, and logical thinking skills. It also demonstrated how programming concepts can be applied to create real-time systems with user interaction.

Overall, this project enhanced programming confidence and provided a strong foundation for developing more advanced graphical applications and games in the future.

09. Future Scopes and Application

The future improvements and applications of this project include:

- Adding advanced features such as levels, power-ups, and multiple difficulty modes
- Improving graphics by using high-quality textures and animations
- Adding sound effects and background music for better user experience
- Implementing a main menu, pause, and restart functionality
- Introducing a high-score system with file handling for saving and retrieving scores
- Enhancing performance using advanced programming techniques
- Extending the game to multiplayer mode or mobile platforms
- Creating a more user-friendly interface with better visual design

This project can be extended to real-world applications such as game development, simulation systems, and interactive software. It provides a strong foundation for developing more complex graphical applications using C++.

References

(1) B.M. Harwani, *Practical C Programming*, Packt Publishing

(2) SFML Official Documentation, Available at: https://www.sfml-dev.org

(3) SFML Tutorials, Available at: https://www.sfml-dev.org/tutorials/

Annexure

CAR.CPP

```
#include "Car.hpp"
#include <iostream>
Car::Car() : speed(0.0f), active(true) { }
void Car::render(sf::RenderWindow& window) {
   if (active) {
      window.draw(sprite) }}
bool Car::loadTexture(const std::string& filename) {
   if (!texture.loadFromFile(filename)) {
      std::cerr << filename << std::endl;
      return false; }
   sprite.setTexture(texture);
   sprite.setOrigin(texture.getSize().x / 2.0f, texture.getSize().y / 2.0f);
   return true; }
void Car::setPosition(float x, float y) {
   sprite.setPosition(x, y); }
sf::Vector2f Car::getPosition() const {
   return sprite.getPosition(); }
sf::FloatRect Car::getBounds() const {
   return sprite.getGlobalBounds(); }
void Car::setSpeed(float s) {
   speed = s; }
float Car::getSpeed() const {
   return speed; }
bool Car::isActive() const {
   return active; }
void Car::setActive(bool state) {
   active = state; }
```

Explanation:

- Base class for all cars
- Stores car properties like sprite, speed, and active state
- Loads texture and applies it to the car sprite
- Sets and gets position of the car

- **ENEMYCAR.cpp**

```
#include "EnemyCar.hpp"
EnemyCar::EnemyCar() : screenBottom(600.0f) {
  speed = 200.0f; }
void EnemyCar::setScreenBottom(float bottom) {
  screenBottom = bottom; }
void EnemyCar::update(float deltaTime) {
  if (!active) return;
  sf::Vector2f pos = sprite.getPosition();
  pos.y += speed * deltaTime;
  sprite.setPosition(pos);
  if (pos.y - sprite.getGlobalBounds().height / 2.0f > screenBottom) {
    active = false;  } }
```

Explanation:

- Represents enemy cars in the game
- Initializes enemy speed and screen boundary
- Moves the enemy car downward continuously

PLAYER.cpp

```
#include "PlayerCar.hpp"
PlayerCar::PlayerCar() : leftBound(0.0f), rightBound(800.0f) {
  speed = 400.0f; }
void PlayerCar::setBounds(float left, float right) {
  leftBound = left;
  rightBound = right; }
void PlayerCar::update(float deltaTime) {
  if (!active) return;
 sf::Vector2f pos = sprite.getPosition();
  float width = sprite.getGlobalBounds().width;
  if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left) || sf::Keyboard::isKeyPressed(sf::Keyboard::A)) {
    pos.x -= speed * deltaTime; }
  if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right) || sf::Keyboard::isKeyPressed(sf::Keyboard::D)) {
    pos.x += speed * deltaTime; }
```

```
  if (pos.x - width / 2.0f < leftBound) {
    pos.x = leftBound + width / 2.0f; }
  if (pos.x + width / 2.0f > rightBound) {
    pos.x = rightBound - width / 2.0f; }
  sprite.setPosition(pos); }
```

Explanation

- Represents the player-controlled car
- Initializes movement speed and road boundaries
- Takes keyboard input (Left/Right or A/D keys)
- Moves the car horizontally based on input
- Uses deltaTime for smooth movement
- Updates the car position on the screen

GAME.cpp

Main Logic

```
#include"Game.hpp"
Game::Game(){
window.create(sf::VideoMode(800,600),"CarDodgingGame");
window.setFramerateLimit(60);    }
voidGame::run(){
while(window.isOpen())
processEvents();
update();
render(); } }
```

Collision Detection

```
bool Game::checkCollision(const Car& a, const Car& b) {
  return a.getBounds().intersects(b.getBounds()); }
```

Enemy Spawn Logic

```
void Game::spawnEnemy() {
  auto enemy = std::make_unique<EnemyCar>();
  enemy->setPosition(rand() % 600, -100);
  enemies.push_back(std::move(enemy)); }
```

Explanation:

- Checks collision between two cars
- Compares overlapping areas of both cars
- Returns true if collision occurs

Main.cpp

```
#include "Game.h"

#include <ctime>

int main(){

  srand(static_cast<unsigned>(time(nullptr)));

  Game game;

  game.run();

  return 0; }
```

Explanation

- Entry point of the program
- Creates the Game object
- Calls the main game loop using run() function
- Starts and controls the execution of the game